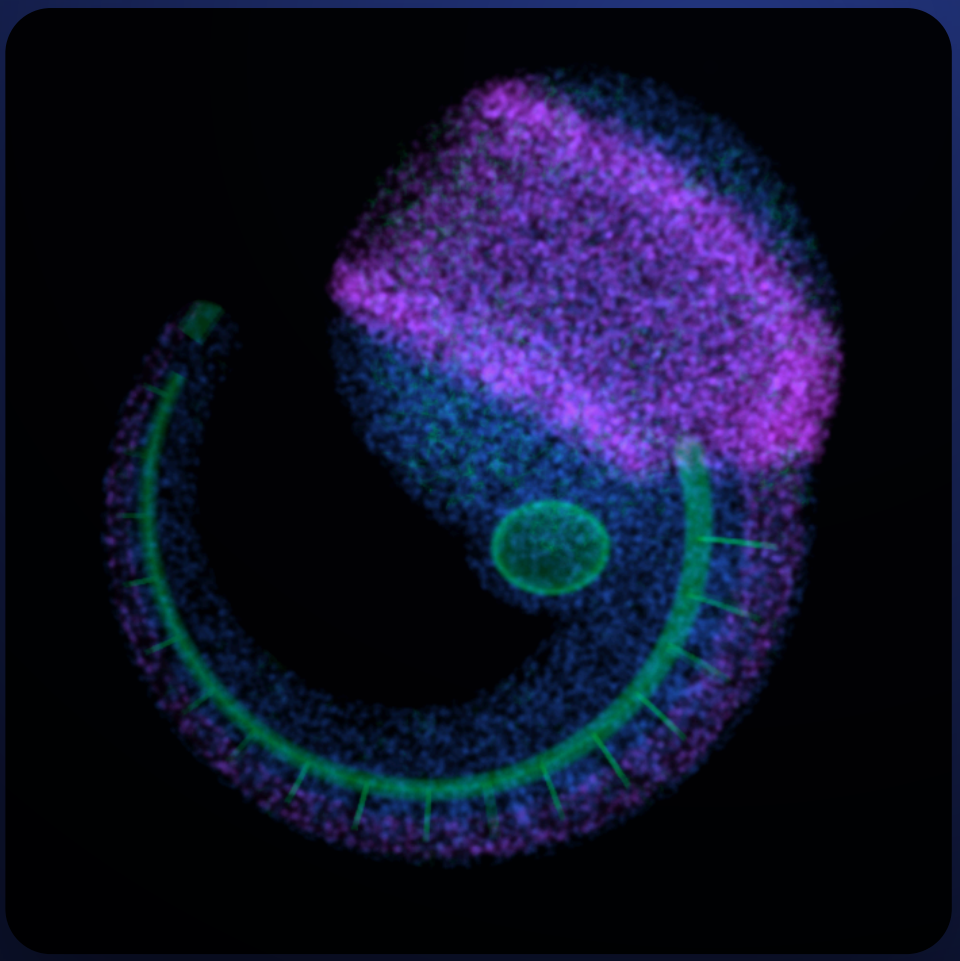


IRIBHM · ULB — LUMEN3D

Lumen3D — l’essentiel

Ce que devient votre fichier Imaris, du microscope à l’écran — et comment bien s’en servir

Le guide du biologiste : préparer son acquisition, la faire traiter, la mettre en ligne, l’explorer en 3D, mesurer, faire des figures, et savoir interpréter ce que l’on voit. Chaque section renvoie au chapitre de la documentation complète (« Lumen3D, de A à Z ») qui donne tous les détails.

Plateforme Web 1.76.0 · Pipeline 0.23.0 · Format de données 6

Novembre 2026

Sommaire

1. Lumen3D en deux pages

1.1 Pourquoi une préparation est indispensable

1.2 Trois types de jeux de données

2. Avant le pipeline : préparer son acquisition

2.1 Le nom du fichier : stade et embryon

2.2 Ce qu'Imaris doit contenir

2.3 Couleurs, canaux, données 8 ou 16 bits

2.4 Timelapse avec suivi cellulaire

3. Lancer le pipeline

4. Le nettoyage de l'image

4.1 Les cinq repères, en une phrase chacun

4.2 Un exemple complet, voxel par voxel

4.3 Sur une vraie coupe

4.4 La limite à connaître : la taille minimale

4.5 Pour un timelapse : une fenêtre commune

5. Réduire la taille sans perte

5.1 La pyramide

5.2 Compression sans perte

5.3 Les briques vides ne sont pas stockées

6. Les briques et les formats

6.1 Format 6 : une copie pour voir, une copie pour exporter

6.2 Format 5 : une matrice résolution × compression (anciens jeux)

7. La fin du pipeline

8. Mettre en ligne et régler le dataset

8.1 Les réglages qui comptent pour vous

9. Explorer en 3D

9.1 Qualité et chargement

9.2 Comment l'image est calculée

9.3 Les trois modes de rendu

10. Régler l'affichage d'un canal

11. Mesurer et explorer l'intérieur

12. Faire des figures et exporter

13. Timelapses et suivi cellulaire

14. Les photographies 2D

15. Interpréter correctement ce que l'on voit

15.1 Ce que la plateforme ne fait pas

16. Que se passe-t-il si… ?

17. Les mots à connaître

1. Lumen3D en deux pages

⏱ EN 30 SECONDES

- Lumen3D affiche des volumes confocaux de **plusieurs gigaoctets** dans un simple navigateur, sans rien installer.
- Un **pipeline** prépare chaque fichier Imaris **une seule fois** ; le fichier d'origine n'est jamais modifié.
- Le **viewer** ne télécharge que ce dont l'écran a besoin, puis calcule l'image 3D sur la carte graphique.



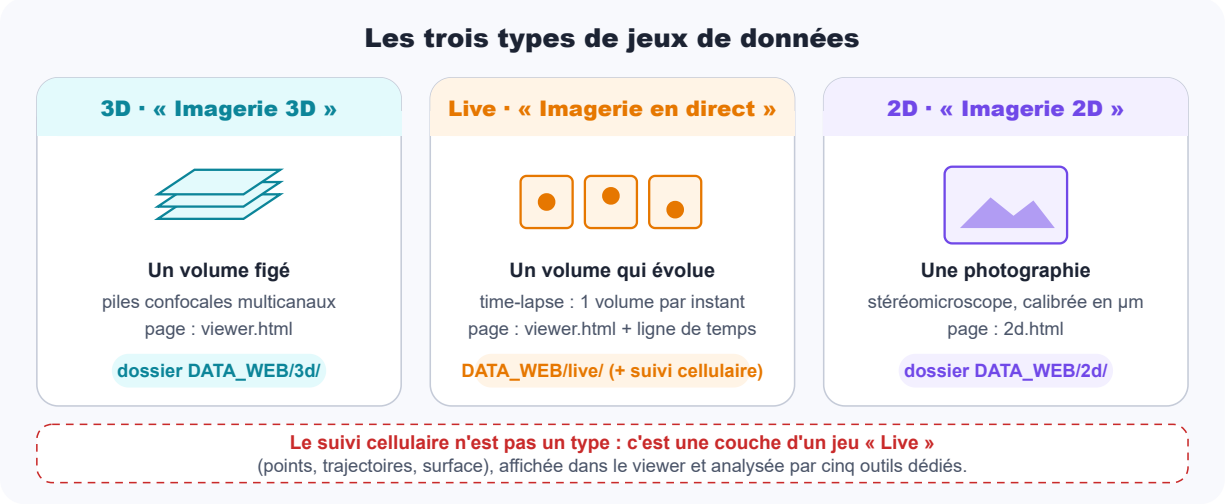
▲ Du microscope à l'écran, en quatre étapes.

1.1 Pourquoi une préparation est indispensable



▲ Même réduit à 1 octet par voxel, un gros embryon dépasse la mémoire d'un ordinateur courant : il faut le découper et n'en charger que la partie utile.

1.2 Trois types de jeux de données



▲ Trois types, trois pages d'affichage. Les noms affichés (3D, 2D, Live) peuvent être renommés par l'administrateur.

TYPE	CE QUE C'EST	PAGE	PARTICULARITÉ
3D	une pile confocale, jusqu'à 4 canaux affichés	viewer 3D	outils de mesure, coupes, Studio
Live	une série temporelle (timelapse)	viewer 3D + frise	peut porter un **suivi cellulaire**
2D	une photographie calibrée (stéréomicroscope)	page 2D	isolation de la coloration X-gal

ℹ️ **À SAVOIR**

Ce document est le résumé de la **documentation complète** (« Lumen3D, de A à Z », 21 chapitres). Les renvois **→ ch. N** pointent vers le chapitre qui explique tout en détail. Les images de volumes proviennent d'**embryons de démonstration synthétiques**, traités par le vrai pipeline.

2. Avant le pipeline : préparer son acquisition

⏱ **EN 30 SECONDES**

- Le **nom du fichier** donne le stade et le numéro d'embryon : choisissez-le bien.
- La **calibration** (taille du voxel) et le **nom des canaux** sont lus dans le fichier Imaris : vérifiez-les avant d'exporter.
- Pour un timelapse suivi, préparez la **source du suivi** (un des trois formats acceptés).

2.1 Le nom du fichier : stade et embryon

Le pipeline lit le stade (`E…`) et l'embryon (`Em…`) dans le nom. Les règles exactes :

DANS LE NOM	STADE LU	EMBRYON LU
`…-E8.5-Em1-…` **ou** `…-E8,5-…`	E8.5	Em1
`…-E8-5-…` **(tiret : 25, 5 ou 75 seulement)**	E8.5	—
`…-E85-…` · `…-E105-…` · `…-E95-…` **(forme compacte)**	E8.5 · E10.5 · E9.5	—
`…-E8-1-DAPI…` **(le « 1 » n'est pas une fraction)**	E8	Em1
`…-E95-2-…`	E9.5	Em2
aucun `E…` **reconnu**	« Unknown »	—

Le nom du dossier publié est le nom du fichier, nettoyé (caractères autres que lettres, chiffres, `.` , `_` , `-` remplacés par `-`). Tout cela se corrige ensuite dans l'administration. → ch. 8

2.2 Ce qu'Imaris doit contenir

Vérifiez dans Imaris avant l'export :

- **la calibration** (taille du voxel en X, Y, Z) : sans elle, pas de barre d'échelle ni de mesure en µm ;
- **le nom de chaque canal** (DAPI, Pecam1…) : un nom vide devient « Channel 1 », « Channel 2 »… ;
- pour un timelapse, **les heures** de chaque instant : l'intervalle retenu est la médiane des écarts.



▲ La taille du voxel est calculée : étendue ÷ nombre de voxels, axe par axe. Ici 921,6 µm ÷ 768 = 1,2 µm.

🔢 **MICRO-EXEMPLE**

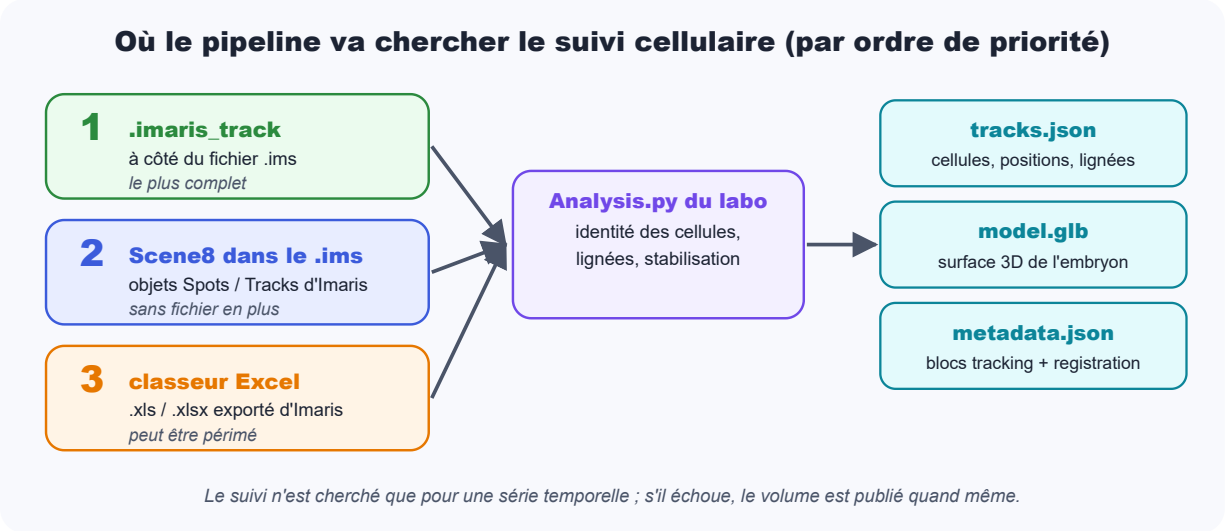
Le pipeline lit l'**étendue physique** (ExtMin → ExtMax) et la divise par le nombre de voxels. 921,6 µm sur 768 voxels → **1,2 µm** par voxel en X. 336 µm sur 112 coupes → **3,0 µm** en Z. Les unités nm et mm sont converties en µm. Une calibration absente n'est **jamais inventée**.

2.3 Couleurs, canaux, données 8 ou 16 bits

- Les **couleurs** d'Imaris ne sont pas reprises : chaque canal reçoit une couleur par défaut (vert, bleu ciel, magenta, rouge…), que l'administrateur change ensuite.
- Fichiers **8 ou 16 bits** : les deux sont acceptés. L'image publiée est toujours en 8 bits (§ 4).
- Plus de **4 canaux** : tous sont traités, mais le viewer n'en affiche que les 4 premiers.

2.4 Timelapse avec suivi cellulaire

Le pipeline cherche le suivi automatiquement, **dans cet ordre** :



▲ Les trois sources possibles, de la plus complète à la plus fragile.

> ⚠️ **ATTENTION**
>
> Le classeur Excel exporté d'Imaris peut être **périmé** (mesuré sur un jeu du laboratoire : 155 des 172 spots). Son nom doit contenir l'intervalle entre deux images (par ex. `30min`) : l'analyse le lit dans le nom. Préférez les objets *Spots/Tracks* enregistrés dans le `.ims` lui-même.

3. Lancer le pipeline

> ⏱ **EN 30 SECONDES**
>
> - Tout se lance avec `RUN.bat`, sur un PC Windows, depuis le **pack Pipeline** téléchargé dans l'onglet *Pipeline* de l'administration.
> - Les `.ims` vont dans `input\`, le résultat sort dans `output\`.
> - Le dataset est construit à part et n'apparaît qu'une fois **entièrement** terminé.



▲ De votre ordinateur au site : déposer, lancer, puis copier ou glisser.

CHOIX DU MENU `RUN.BAT`	CE QUE CELA FAIT
`[1]`	Traiter des volumes Imaris (`.ims` → `output\` , suivi cellulaire inclus)
`[2]`	Analyse de tracking Imaris (Excel → `tracking\OUTPUT\`)
`[3]`	Importer des photographies 2D (`.tif` → `output\2d\`)
`[4]`	Attacher un suivi à un dataset déjà traité
`[5]`	Vérifier l'environnement seulement

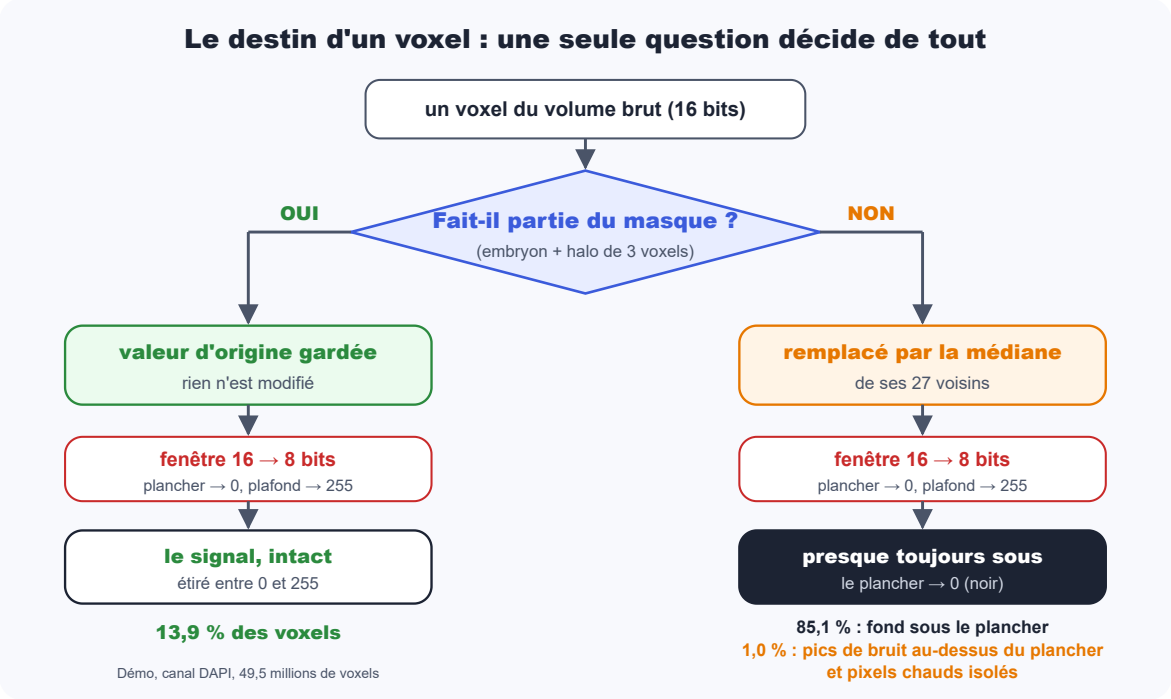
👉 **ASTUCE**

Deux éditions du pack : **légère** (~3 Mo, télécharge Python au premier lancement) et **complète hors-ligne** (~70 Mo, tout est embarqué). Les versions des bibliothèques sont figées : deux ordinateurs produisent les **mêmes octets** à partir du même `.ims` . Retraiter un dataset déjà publié **conserve** vos réglages (nom, couleurs, orientation, vue par défaut, galerie, visibilité). → ch. 8

4. Le nettoyage de l'image

⏱ **EN 30 SECONDES**

- Le pipeline pose **une seule question** à chaque voxel : *fait-il partie de l'embryon ?*
- **Oui** → sa valeur n'est **pas modifiée**. **Non** → il est remplacé par la **médiane** de ses voisins, ce qui le ramène presque toujours à **0**.
- Puis tout passe en **8 bits** par une simple règle de trois : le fond devient exactement **0**.



▲ Le destin d’un voxel. Pourcentages mesurés sur tout le volume du jeu de démonstration (canal DAPI).

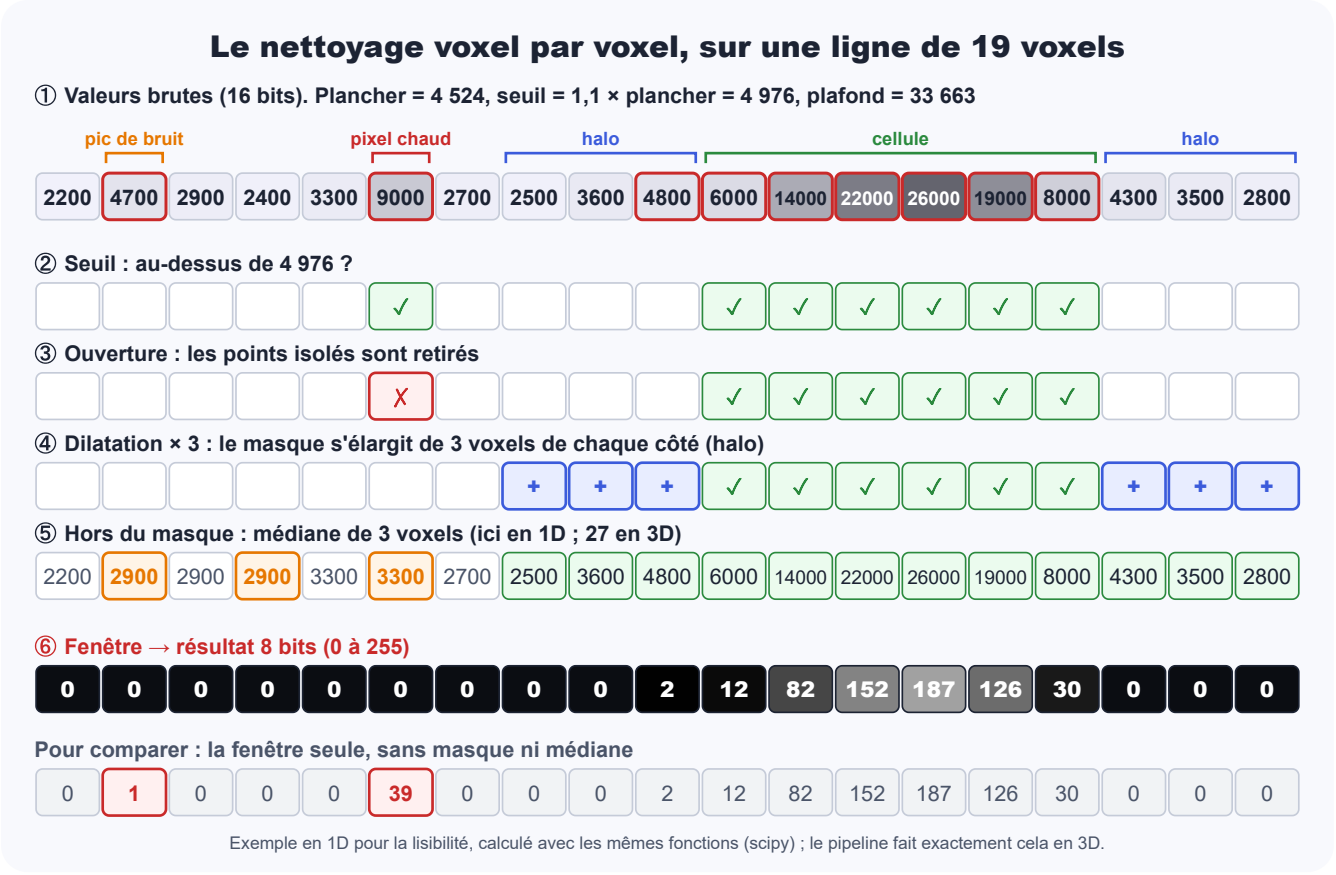
4.1 Les cinq repères, en une phrase chacun

	QUESTION	RÉPONSE DU PIPELINE
Plancher	À partir d’où est-ce du signal ?	la valeur sous laquelle se trouvent **99 %** des voxels des **8 coins** du volume (là où il n’y a pas d’embryon)
Plafond	Où mettre le blanc ?	la valeur sous laquelle se trouvent **99,9 %** des voxels (1 sur 4 dans chaque direction est examiné)
Masque	Où est l’embryon ?	les voxels **au-dessus de 1,1 × plancher**, **sans les points isolés**, plus un **halo de 3 voxels**
Médiane	Que faire du reste ?	chaque voxel hors masque prend la **valeur du milieu** de ses 27 voisins (cube 3 × 3 × 3)
Fenêtre	Comment passer en 8 bits ?	plancher → **0**, plafond → **255**, une ligne droite entre les deux

🔢 **MICRO-EXEMPLE**

Sur le canal DAPI du jeu de démonstration : plancher = **4 524**, plafond = **33 663**. Un voxel brut de **20 000** devient 255 × (20 000 − 4 524) ÷ (33 663 − 4 524) = **135** (partie décimale tronquée). Un voxel à 4 000 devient **0** ; un voxel à 40 000 devient **255**.

4.2 Un exemple complet, voxel par voxel

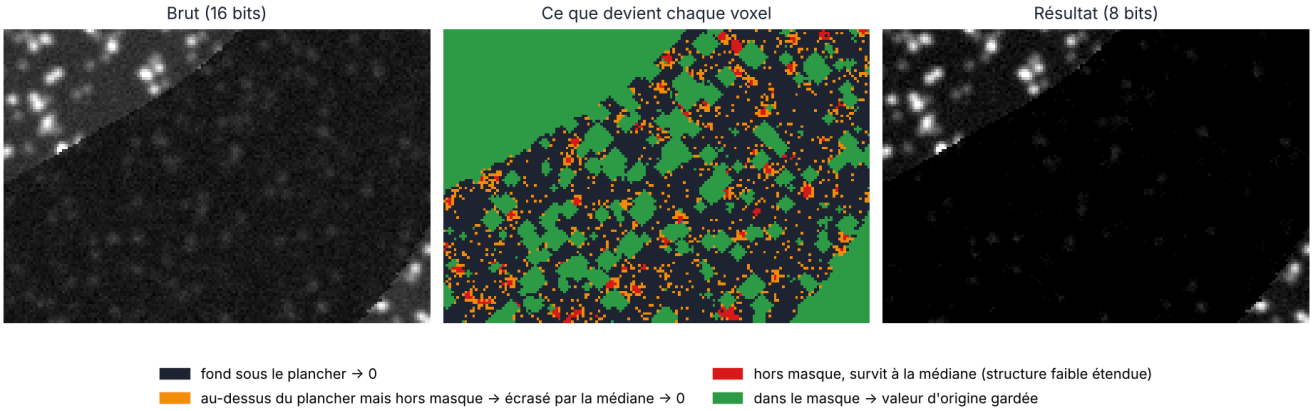


▲ 19 voxels alignés, avec les vrais repères du jeu de démonstration. En une dimension pour pouvoir lire chaque nombre ; le pipeline fait la même chose en 3D.

1. **Seuil** : seuls la cellule et le pixel chaud (9 000) dépassent 4 976. Le pic de bruit (4 700) ne passe pas.
2. **Ouverture** : le pixel chaud est **seul**, il est retiré. La cellule, assez large, reste.
3. **Halo** : le masque s’élargit de 3 voxels autour de la cellule ; son bord faible (4 800) est protégé.
4. **Médiane** hors du masque : le pic de bruit 4 700 → 2 900, le pixel chaud 9 000 → 3 300.
5. **Fenêtre** : tout ce qui est sous le plancher → 0. La cellule va de 12 à 187, son bord faible donne 2.

Dernière ligne de la figure : **sans masque ni médiane**, le pic de bruit et le pixel chaud resteraient visibles (1 et 39). Sur le volume entier, masque + médiane éliminent **97 %** de ces points parasites (341 674 → 10 069). → ch. 5

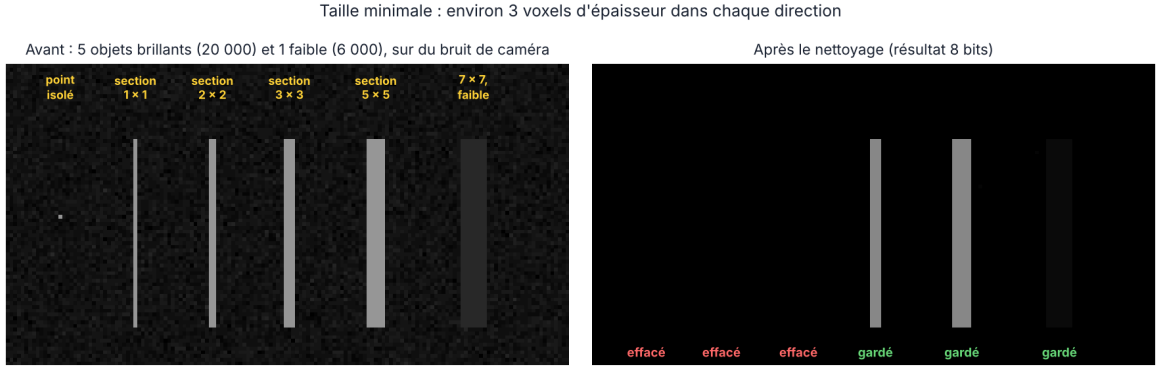
4.3 Sur une vraie coupe



Coupe z = 56, canal DAPI, zone de 160 × 110 voxels où le tissu est faible (jeu de démonstration).

▲ À gauche le brut, au milieu ce que devient chaque voxel, à droite le résultat, dans une zone où le tissu est faible. Vert : gardé tel quel. Orange : écrasé par la médiane. Rouge (rare) : zone faible mais étendue, qui survit.

4.4 La limite à connaître : la taille minimale



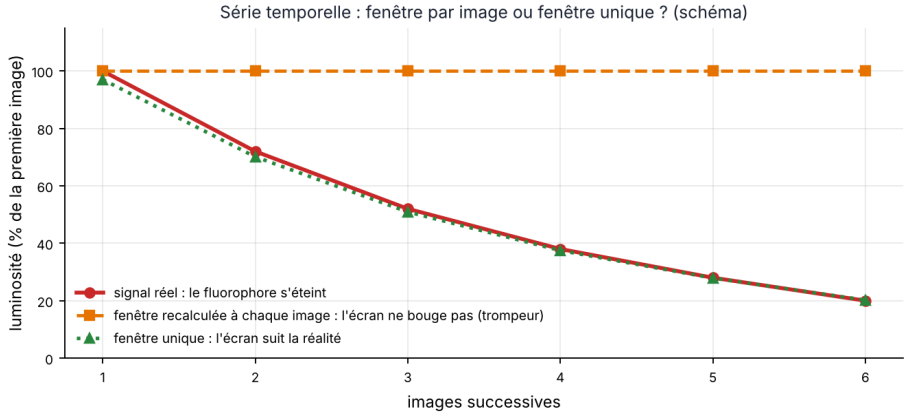
Volume synthétique de test, coupe du milieu : chaque barre a une section carrée (même épaisseur en X et en Z). Même plancher et même plafond que le canal DAPI de démonstration.

▲ Barres de section carrée, brillantes (20 000) ou faibles (6 000). Les sections 1 × 1 et 2 × 2 sont effacées ; à partir de 3 × 3, l'objet est gardé, même faible.

⚠ **ATTENTION**

Un objet plus fin qu'environ 3 voxels dans une direction est effacé, même très brillant (un point, un fil, une feuille d'un voxel d'épaisseur). Avec des voxels de 1,2 × 1,2 × 3 µm, cela fait **3,6 µm en X et Y, 9 µm en Z** : un objet présent sur une ou deux coupes seulement peut disparaître. Calculez avec **vos** tailles de voxel ; en cas de doute, comparez avec le `.ims` d'origine. Un objet fin **collé** (à moins de 3 voxels) à un objet plus épais est protégé par le halo. → ch. 5

4.5 Pour un timelapse : une fenêtre commune



▲ Avec une fenêtre par image, une série qui s'éteint semblerait garder la même intensité. Lumen3D utilise une seule fenêtre pour toute la série (schéma, chiffres illustratifs).

Le plancher et le plafond sont calculés sur **jusqu'à 8 instants** répartis dans la série. Le niveau de signal de chaque instant est **enregistré** dans `metadata.json`, mais aucun réglage du viewer ne l'utilise encore : une série qui s'éteint s'affiche réellement plus sombre.

? POURQUOI CE CHOIX ?

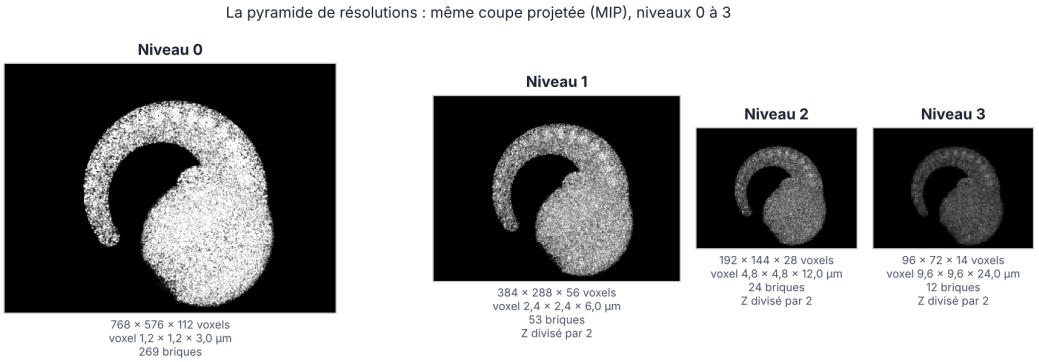
Une méthode de seuillage automatique (Otsu) et un débruitage par réseau de neurones (Noise2Void) ont été essayés puis **retirés** en version 0.12.0 : ils créaient des « taches » colorées artificielles. → ch. 5

5. Réduire la taille sans perte

⏱ EN 30 SECONDES

- **8 bits** au lieu de 16 : deux fois moins d'octets.
- Une **pyramide** de versions de plus en plus petites permet d'afficher vite, puis de préciser.
- Le volume est coupé en **briques** de 64^3 voxels compressées **sans perte** ; les briques vides ne sont **pas stockées**.

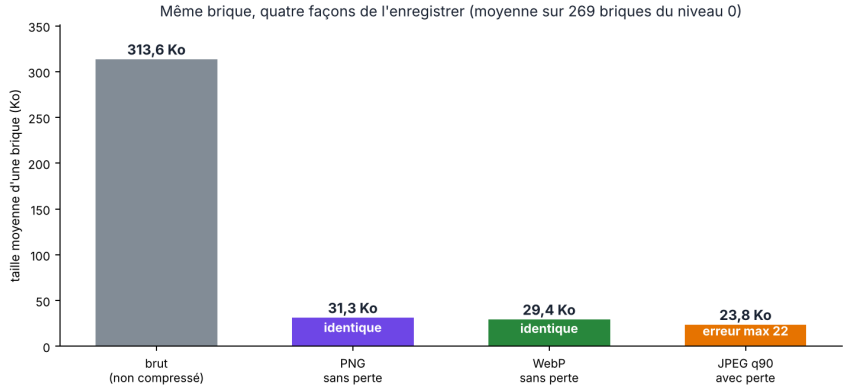
5.1 La pyramide



▲ Les quatre niveaux du jeu de démonstration, à l'échelle. Chaque niveau est la moyenne de blocs 2 × 2 (× 2 en Z quand les voxels sont assez fins en Z) du niveau du dessous.

- X et Y sont divisés par 2 à chaque niveau ; **Z seulement tant que** le voxel en Z ne devient pas plus de 1,5 fois plus grand qu'en XY.
- La moyenne est **entière, arrondie au plus proche**. On s'arrête quand le plus grand côté fait 128 voxels ou moins.
- Le niveau 0 est l'image nettoyée telle quelle, **jamais rééchantillonnée**. → ch. 6
- C'est la règle du **format 4**, celle qu'écrivait le pipeline jusqu'à la version 0.22. Les **formats 5 et 6** gardent toutes les tranches et visent des tailles exactes (512, 256…) : voir § 6.1.

5.2 Compression sans perte



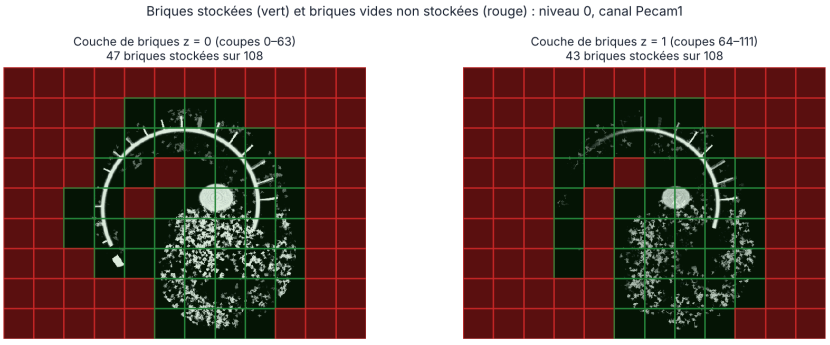
▲ Taille moyenne d'une brique du jeu de démonstration. Seul le JPEG est plus petit… mais il modifie les valeurs.

FORMAT	TAILLE MOYENNE	VALEURS EXACTES ?
Brut	313,6 Ko	oui
PNG (meilleure compression)	31,3 Ko	oui
WebP sans perte (choisi)	**29,4 Ko**	oui
JPEG qualité 90	23,8 Ko	**non** : jusqu'à 22 niveaux d'écart, 11,7 % des voxels modifiés

❓ **POURQUOI CE CHOIX ?**

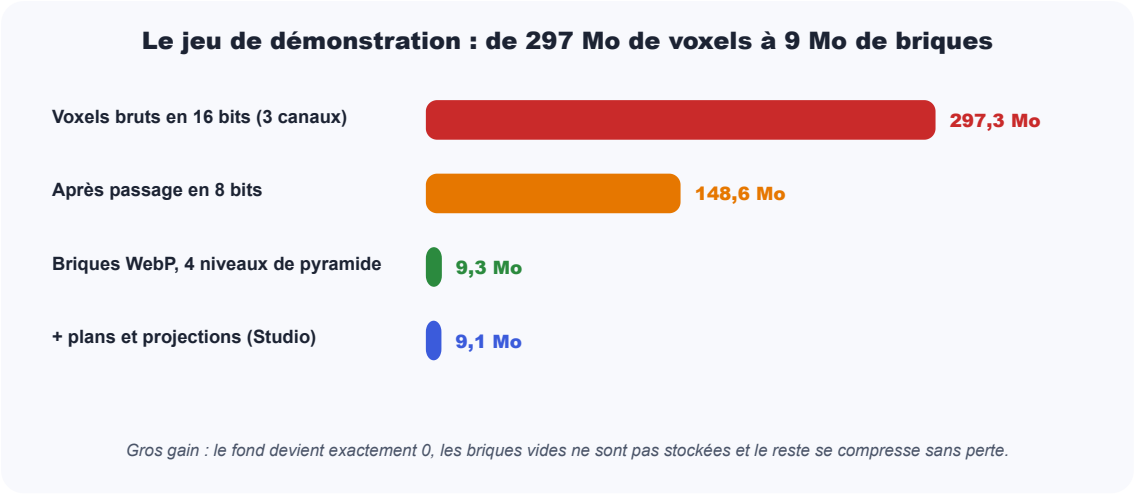
Ce sont des **mesures** : une compression avec perte inventerait du signal (2,2 % des voxels à 0 deviennent non nuls en JPEG). Le WebP est un peu plus compact que le PNG et le navigateur le décode très vite. → ch. 6

5.3 Les briques vides ne sont pas stockées



▲ Vert : brique stockée. Rouge : brique dont tous les voxels valent 0, non stockée. Jeu de démonstration, niveau 0.

Une brique n’est stockée que si **au moins un voxel** de son intérieur vaut 1 ou plus. Sur le jeu de démonstration : **358 briques stockées sur 777** possibles.

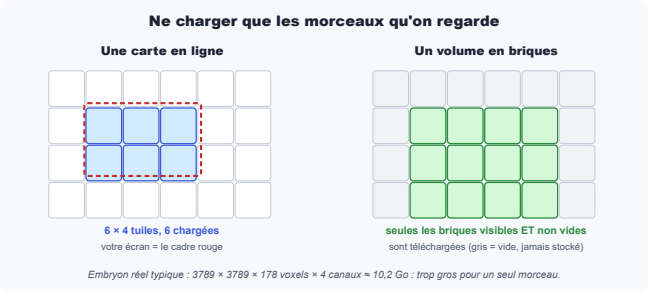


▲ Du fichier d’origine au dossier publié : 297 Mo de voxels bruts deviennent 9 Mo de briques (jeu de démonstration).

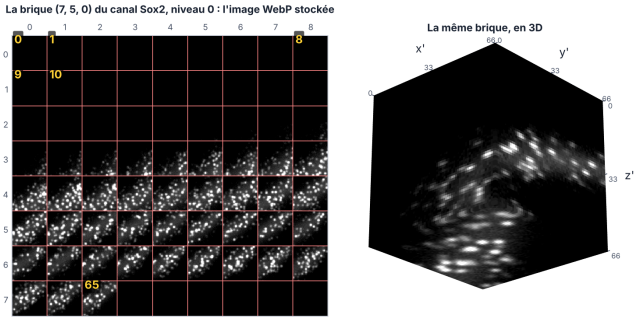
6. Les briques et les formats

⏱ **EN 30 SECONDES**

- Une brique = **64³ voxels** utiles + une **bordure d’un voxel**, rangée à plat dans **une image** WebP.
- Les briques sont regroupées en **paquets** ; un petit **index** dit où trouver chacune.
- Des **plans** XY et des **projections par couche** accélèrent les figures du Studio.



▲ Comme les tuiles d'une carte en ligne : seules les briques visibles et non vides sont téléchargées.



▲ Une vraie brique du jeu de démonstration : ses 66 coupes rangées en 9 × 8 dans une seule image.

- **La bordure** copie le voxel voisin : l'image peut être lissée sans couture visible entre deux briques. → ch. 7
- **Les plans** (`planes/`) : une coupe XY à pleine résolution lit **94 Ko au lieu de 5 078 Ko** sur le jeu de démonstration.
- Les **formats** : un dataset porte un numéro de format (1 à 6, le plus récent est **6**). Le pipeline 0.23.0 écrit le format 6 par défaut. Un jeu plus ancien se met à niveau **sans retraitement**, dans l'onglet *Mises à jour des données* de l'administration. → ch. 17

6.1 Format 6 : une copie pour voir, une copie pour exporter

⏱ **EN 30 SECONDES**

- Chaque niveau est rangé en **deux copies** : **WebP qualité 30** (Q30) pour l'écran, **sans perte** pour les exports et les téléchargements.
- Vous ne choisissez que la **résolution**. Une mesure se prend sur l'image affichée (donc sur la copie Q30 par défaut) ; les exports lisent toujours la copie sans perte, au niveau affiché.
- Un jeu au **format 5** propose cinq compressions à choisir. Une mise à jour des données le convertit en format 6.

USAGE	COPIE LUE
Affichage 3D (par défaut)	WebP Q30
Mesures	l'image affichée : Q30 par défaut, sans perte si **Chargement rapide** est décoché
Figures, exports	sans perte, au niveau affiché
Téléchargement	sans perte (OME-Zarr) ou Q30 (`.q30.zip`)

🔢 **MICRO-EXEMPLE**

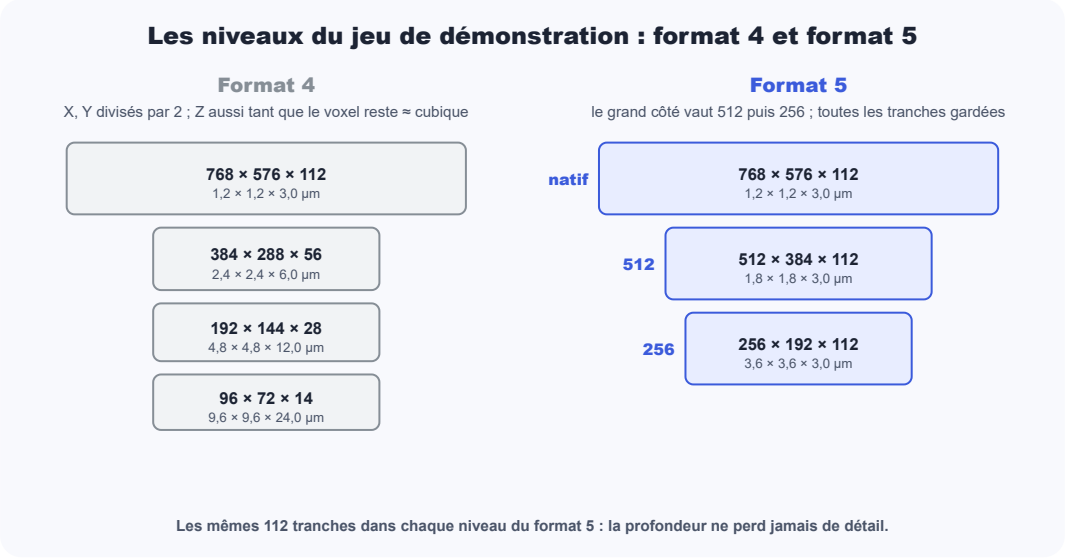
Sur E8.5 Em1 à pleine résolution : **26 Mo** en Q30 contre **89 Mo** sans perte, avec une erreur moyenne de **1,5 niveau de gris sur 255**. La conversion d'un jeu de format 5 en format 6 a ramené ses briques de **834 Mo à 588 Mo**.

6.2 Format 5 : une matrice résolution × compression (anciens jeux)

⏱ EN 30 SECONDES

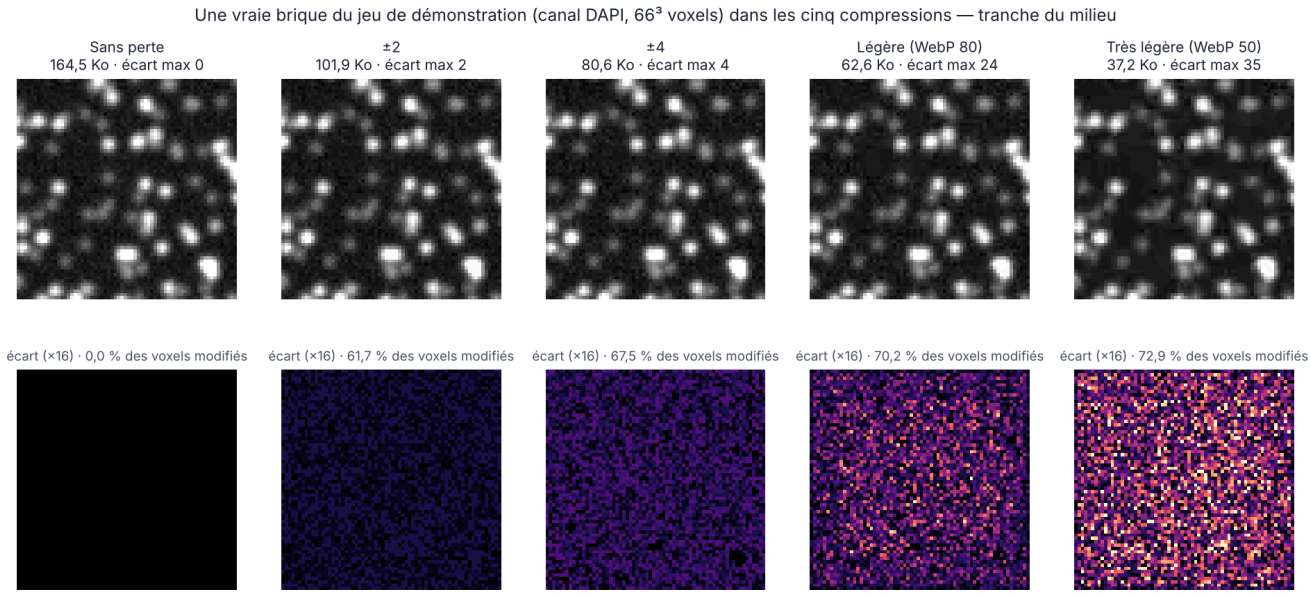
- Le même volume est rangé à **plusieurs résolutions** (les lignes) et en **cinq compressions** (les colonnes). Vous choisissez votre case.
- Les niveaux réduits **gardent chaque tranche** (Z n'est jamais réduit).
- Les **distances** se prennent sur la case affichée, donc elles la suivent. Les **exports** proposent une version **sans perte** (*Max*), quelle que soit la case affichée.

Sur les jeux au **format 5** (encore présents sur les anciens jeux), la qualité se choisit dans une seconde liste, à cinq colonnes :



▲ Les niveaux du jeu de démonstration en format 4 et en format 5 (dimensions réelles).

COLONNE	CE QUI EST GARANTI	POIDS (VS SANS PERTE)
Max (sans perte)	chaque voxel exact	1
Extra (±2)	aucun voxel ne bouge de plus de 2 niveaux de gris (sur 255)	0,62 – 0,82
High (±4)	aucun voxel ne bouge de plus de 4	0,50 – 0,69
Medium (WebP 80)	WebP avec perte (qualité 80) : petits changements visuels	0,35 – 0,61
Low (WebP 50)	WebP avec perte (qualité 50) : lissage visible	0,22 – 0,40



▲ La même vraie brique dans les cinq compressions : en haut la tranche du milieu, en bas l'écart avec l'original (× 16).

- Le **fond reste exactement noir** dans toutes les colonnes : ±2 et ±4 gardent le 0 à part ; les colonnes avec perte remettent à 0 les petites valeurs qui « bavent » (une **zone morte**).
- Trois **copies de coupe** (plans XY, lignes XZ / YZ, petites briques obliques) rendent les coupes natives du Studio rapides : une coupe oblique native a lu 23 Mo au lieu de 60 sur un embryon de 1946² × 72. → ch. 7
- Le prix : **plus de place sur le disque** (9 Mo de briques → environ 80 Mo pour le jeu de démonstration). Ce que chaque visiteur télécharge, lui, diminue.

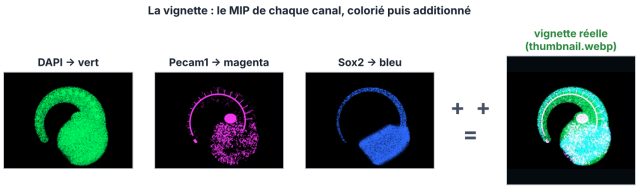
7. La fin du pipeline

⏱ EN 30 SECONDES

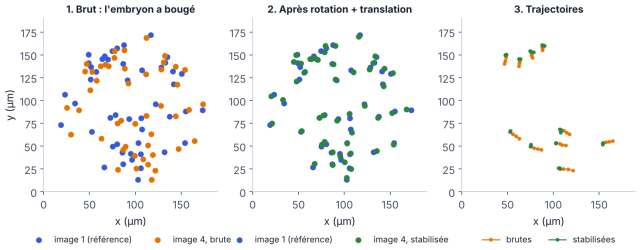
- Le pipeline écrit une **fiche** (`metadata.json`), une **vignette**, les **histogrammes** des canaux.
- Il attache le **suivi cellulaire** d'un timelapse et **stabilise** les positions.
- Il publie **tout ou rien** : jamais de dataset à moitié copié.

FICHIER PUBLIÉ	RÔLE
`metadata.json`	la fiche : nom, stade, dimensions, calibration, canaux, format
`thumbnail.webp`	la vignette de l'explorateur (projection maximale en fausses couleurs)
`bricks/`	les briques de tous les niveaux et leur index
`planes/` , `mips/`	plans XY et projections par couche de 64 plans

FICHIER PUBLIÉ	RÔLE
`tracks.json`, `model.glb`	suivi cellulaire et surface de l'embryon (timelapses)
`download/`	(option) le `.ims` d'origine, un TIFF ImageJ, des projections PNG, un README



▲ La vignette : trois projections maximales coloriées et additionnées.



▲ La stabilisation : une rotation + translation (sans déformation) superpose les cellules de deux instants.

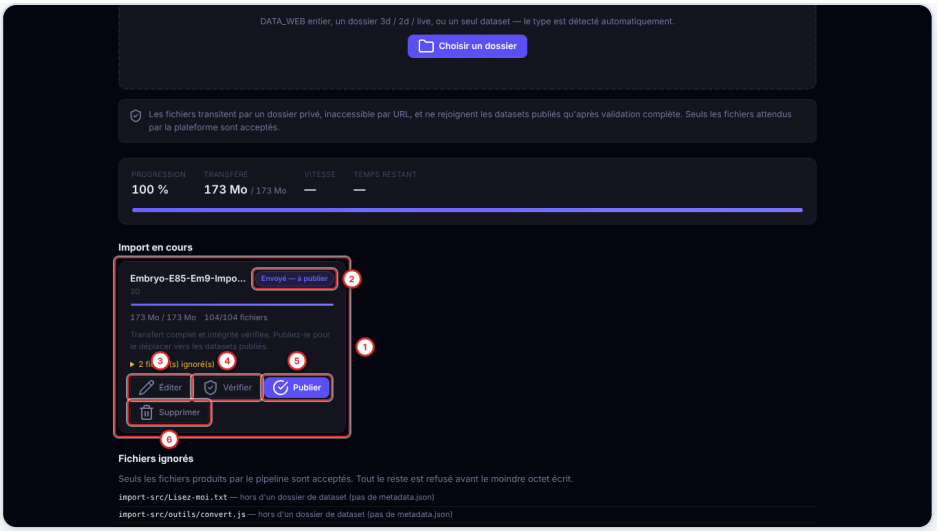
ℹ️ À SAVOIR

Stabilisation. L'embryon bouge pendant l'acquisition. Pour chaque instant, le pipeline calcule le déplacement **rigide** qui superpose au mieux les cellules (méthode de Kabsch, au moins 3 cellules de référence). Le viewer applique ce déplacement **au regard**, pas aux voxels : aucun voxel n'est rééchantillonné. → ch. 8 et 13

8. Mettre en ligne et régler le dataset

⏱ EN 30 SECONDES

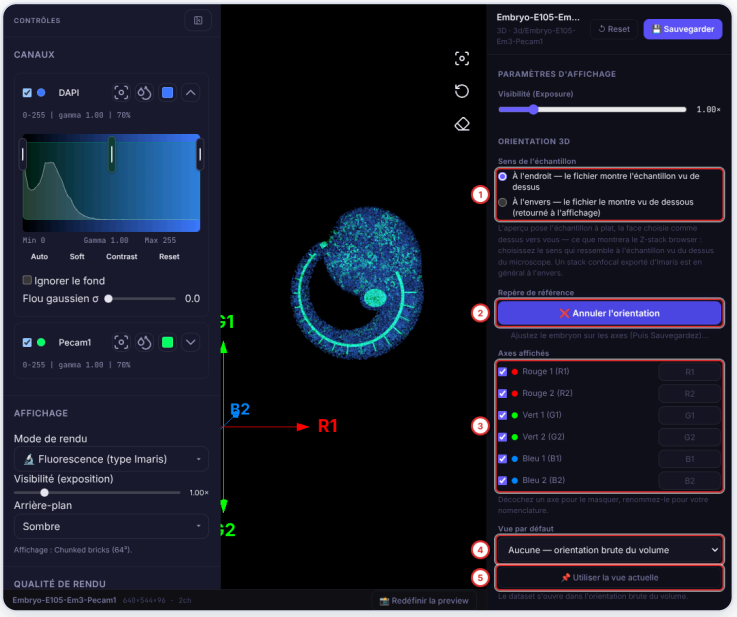
- Deux voies : **copier** le dossier dans `DATA_WEB/` du serveur, ou le **glisser** dans l'onglet *Import* de l'administration.
- Un dataset importé est **masqué** jusqu'à ce que vous l'affichiez.
- L'administration règle le nom, le stade, les couleurs, la calibration, l'**orientation** et la **vue par défaut**.



▲ L'onglet Import : un dataset entièrement transféré et vérifié, prêt à publier (boutons Éditer, Vérifier, Publier, Supprimer).

- Le transfert **reprend tout seul** après une coupure : il suffit de reglisser le même dossier (pendant 7 jours).
- L'ordre d'envoi est pensé pour vous : la fiche, la vignette et le **niveau le plus grossier** partent en premier, si bien que le dataset est **réglable en quelques minutes**, pendant que le reste arrive.
- Seuls les fichiers produits par le pipeline sont acceptés. → Guide de l'administrateur, ch. 4
- Deux onglets surveillent le serveur : **Stockage** (la place prise sur le disque) et **Intégrité des données** (un contrôle de chaque dataset). Les textes de l'onglet Stockage apparaissent pour l'instant en français, dans toutes les langues de l'interface, faute de traduction.

8.1 Les réglages qui comptent pour vous



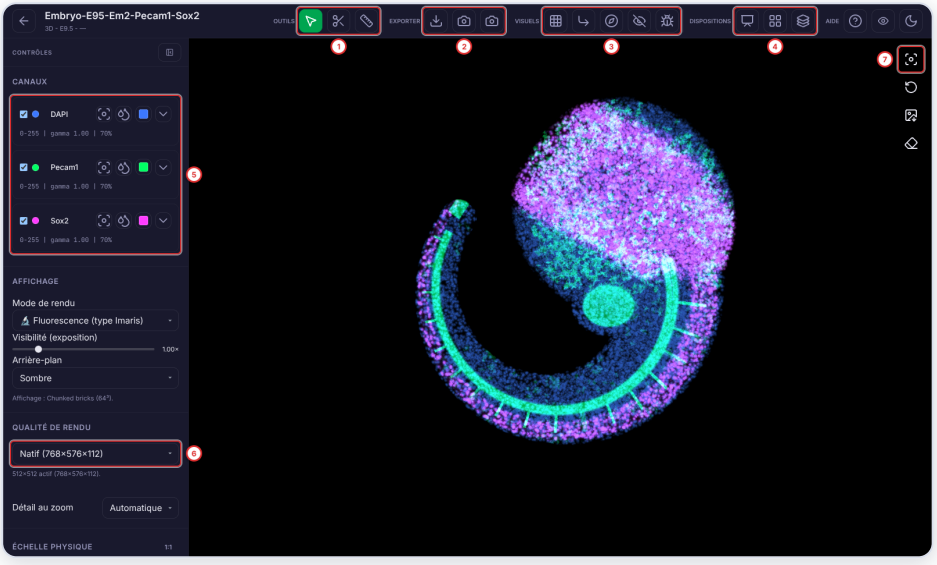
▲ L'orientation dans l'administration : sens de l'échantillon, repère des axes, axes affichés, vue par défaut.

RÉGLAGE	À QUOI IL SERT
Nom, stade, embryon, description	ce que lisent les visiteurs dans l’explorateur
Couleurs des canaux	la couleur par défaut de chaque canal à l’ouverture
Calibration	corriger une taille de voxel absente ou fausse
Sens de l’échantillon	« À l’envers » pour une pile vue de dessous (cas fréquent d’un export Imaris) : le navigateur Z-stack montrera la bonne face
Définir l’orientation	aligner les axes rouge / vert / bleu sur l’anatomie, puis les renommer (antérieur, dorsal…)
Vue par défaut	la pose dans laquelle le dataset s’ouvre
Galerie	joindre des figures annotées, visibles dans le viewer
Visibilité	afficher ou masquer le dataset dans l’explorateur public

9. Explorer en 3D

⏱ EN 30 SECONDES

- Ouvrez un dataset depuis l’**explorateur** (recherche, filtres par type et par stade).
- Le viewer affiche d’abord une **image grossière**, puis le détail arrive **en partant du centre**.
- Trois **modes de rendu** pour trois questions différentes.

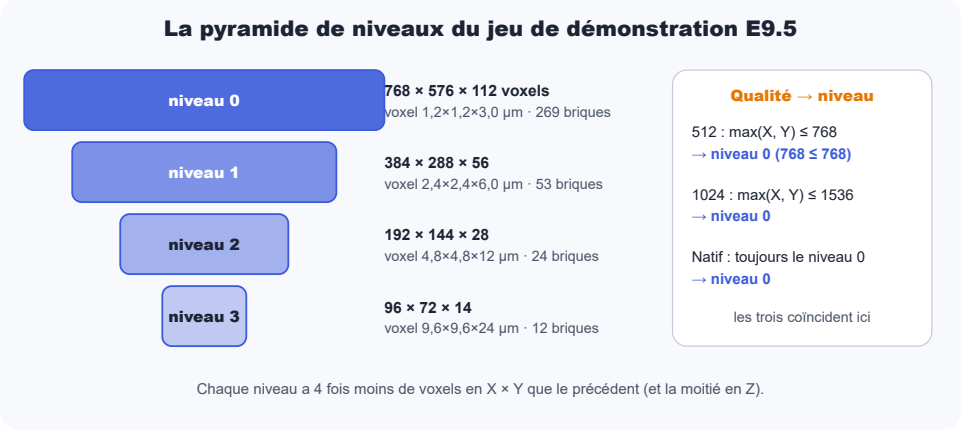


▲ Le viewer : barre d’outils en haut, canaux à gauche, le volume au centre (jeu de démonstration).

9.1 Qualité et chargement

⏱ EN 30 SECONDES

- Sur un jeu de **format 6** (le plus récent), vous choisissez une **résolution** ; l'image est affichée en **WebP Q30**.
- **Chargement rapide** est **coché par défaut**. Le décocher affiche les résolutions **sans perte**, avec des téléchargements plus gros.
- La **mémoire graphique** est détectée toute seule : une résolution trop lourde pour la carte affiche le niveau plus grossier, avec un message.



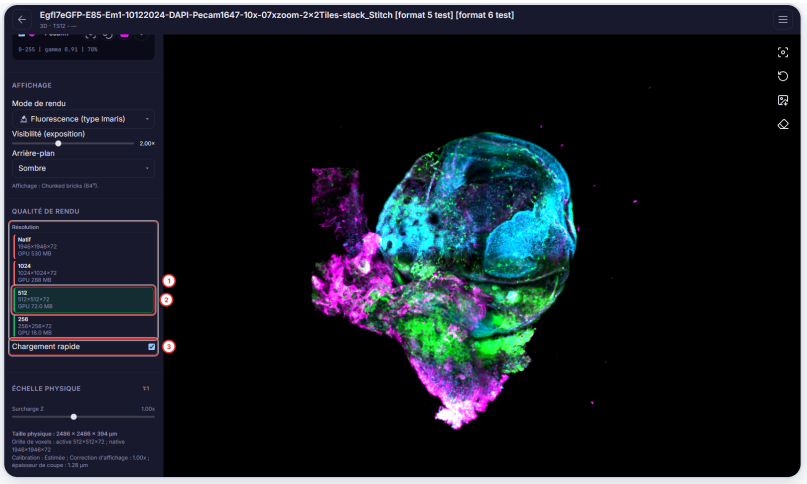
▲ La pyramide réelle du jeu de démonstration et la règle des trois qualités.

Ouvrez **Qualité de Rendu** dans le panneau latéral. Les noms des résolutions :

RÉSOLUTION	NIVEAU CHARGÉ
512 (par défaut)	le niveau le plus fin dont le grand côté XY ne dépasse pas **768** px
1024	le niveau le plus fin dont le grand côté ne dépasse pas **1 536** px
Native	le niveau 0, pleine résolution

Sur un jeu de **format 6**, la liste ne contient que les **résolutions**. Chaque ligne indique la **mémoire graphique** que prend le niveau une fois chargé :

COULEUR	SIGNIFICATION
vert	la carte tient le niveau à l'aise (moins de 75 % du budget)
orange	proche du budget (75 à 100 %)
rouge	au-delà du budget : le viewer affiche le niveau plus grossier suivant



▲ La liste de qualité d’un jeu de format 6 : les résolutions, leur mémoire graphique et Chargement rapide.

N	CE QUE C’EST
1	la liste des résolutions, avec la mémoire graphique que prend chaque niveau
2	la résolution affichée (ici 512 : 512 × 512 × 72 voxels)
3	**Chargement rapide**, coché par défaut

ℹ️ À SAVOIR

Chargement rapide (case sous la liste, cochée par défaut). **Coché** : l’image vient des briques WebP Q30, les plus légères. **Décoché** : les mêmes résolutions viennent des briques sans perte, plus lourdes à télécharger. Ce choix est gardé dans ce navigateur. Décochez-le si vous voulez afficher les données exactes, sans compression d’affichage. Les exports restent sans perte dans les deux cas ; les mesures suivent l’image affichée.

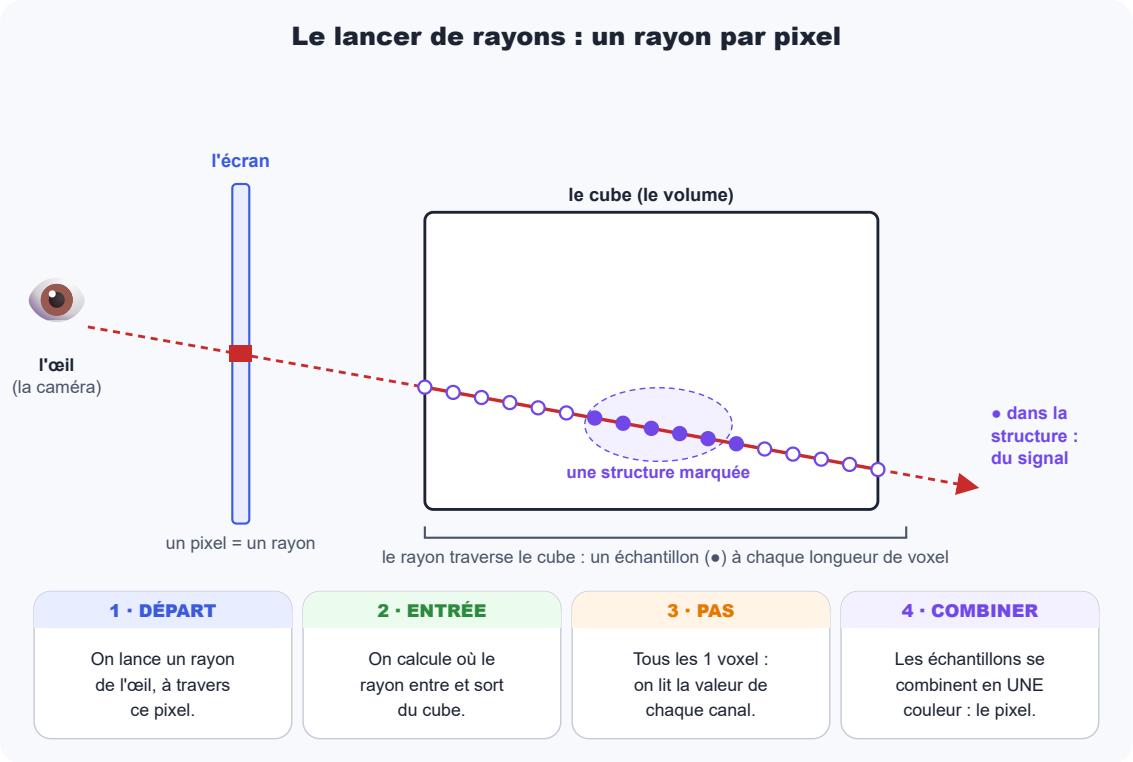
Sur un jeu de **format 5**, une seconde liste, **Qualité**, propose les cinq compressions du § 6.1 (Max, Extra, High, Medium, Low). Le bouton **Comment choisir ?** ouvre un guide : la taille à télécharger de chaque case, le temps à 5, 20 ou 100 Mb/s, et la mémoire graphique. Son conseil : **la résolution la plus fine affichée en vert, en qualité Extra** ; pour une mesure ou une figure, **Max en Native**.

🦴 POUR LES CURIEUX

La mémoire graphique est cherchée d’après le nom du GPU fourni par le navigateur, dans une table des cartes NVIDIA, AMD et Intel Arc. Le viewer en prend **60 %**, laisse **2 Gio** au navigateur et au système, et plafonne à **16 Gio** (plancher : 1 Gio). Une carte absente de la table reçoit une estimation par catégorie. Chaque perte de contexte graphique de la semaine écoulée réduit le budget de moitié. Sur les GPU de téléphone et de tablette (Adreno, Mali…) et dans les navigateurs Android, la compression n’est pas utilisée : les voxels exacts sont affichés.

Si la carte graphique n’a pas assez de mémoire, le viewer prend **le niveau suivant plus léger** et le dit dans un message. Une ligne de progression indique la qualité, le pourcentage et l’étape en cours. → ch. 10

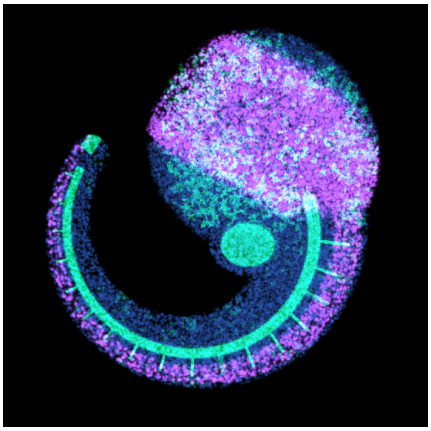
9.2 Comment l’image est calculée



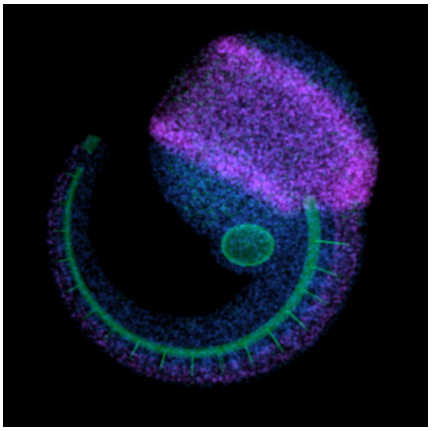
▲ Un rayon par pixel : il entre dans le volume, prend un échantillon par voxel traversé, et combine ces valeurs.

L’image n’est pas une photo : la carte graphique la **recalcule** à chaque affichage, des milliers de pixels en parallèle (Three.js + WebGL2). Elle est **la même en mouvement et à l’arrêt** : pleine résolution, un échantillon par voxel. Les cellules vides sont sautées. En mode *Fluorescence (type Imaris)*, les réglages de couleur et de fenêtre se recalculent instantanément (§ 10). → ch. 9

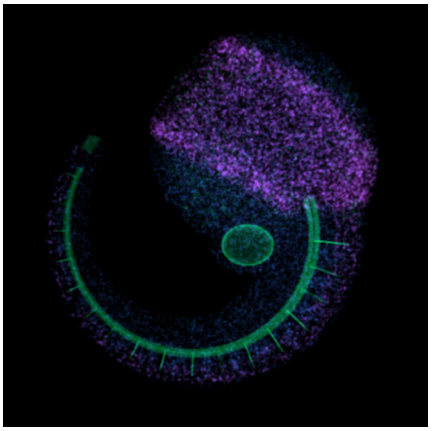
9.3 Les trois modes de rendu



▲ Fluorescence (type Imaris), mode par défaut.



▲ Fluorescence naturelle.



▲ Structure (DVR).

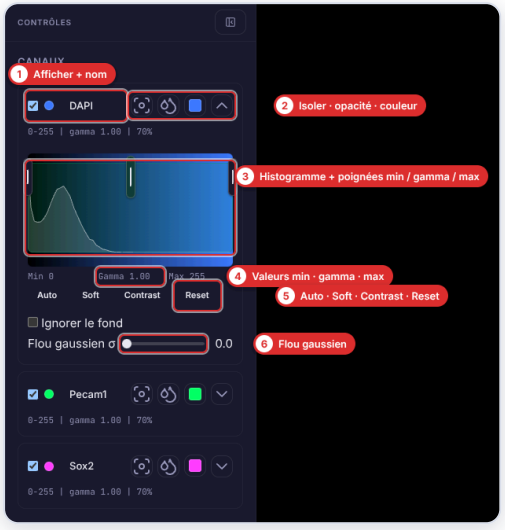
MODE	CE QUE VOUS VOYEZ	BON POUR…
Fluorescence (type Imaris)	le **maximum** de chaque canal le long du rayon, couleurs additionnées, sans profondeur	voir tout le signal, comme une MIP

MODE	CE QUE VOUS VOYEZ	BON POUR…
Fluorescence naturelle	chaque fluorophore brille ; ce qui est dense **cache** ce qui est derrière	percevoir la forme et la profondeur
Structure (DVR)	une opacité accumulée de l’avant vers l’arrière	des surfaces et des volumes pleins

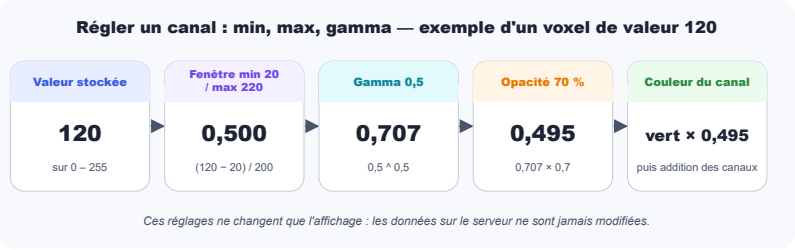
10. Régler l’affichage d’un canal

⏱ EN 30 SECONDES

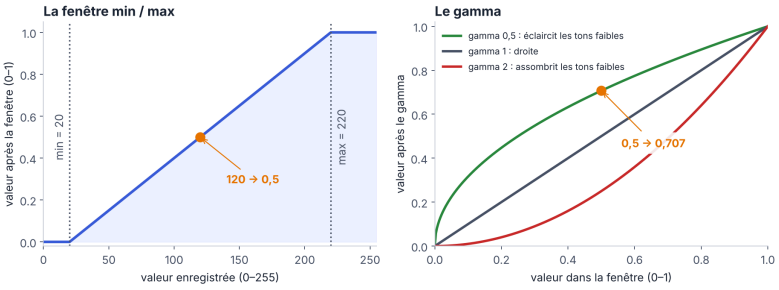
- Chaque canal a un **min**, un **max**, un **gamma**, une **opacité** et une **couleur** ; les canaux sont **additionnés**.
- L’**histogramme** montre la répartition des valeurs 0–255 du niveau le plus grossier, en 64 barres.
- Ces réglages ne changent **que l’affichage**, jamais les données.



▲ La carte d’un canal dépliée.



▲ Le calcul d’un voxel, du stockage à l’écran.

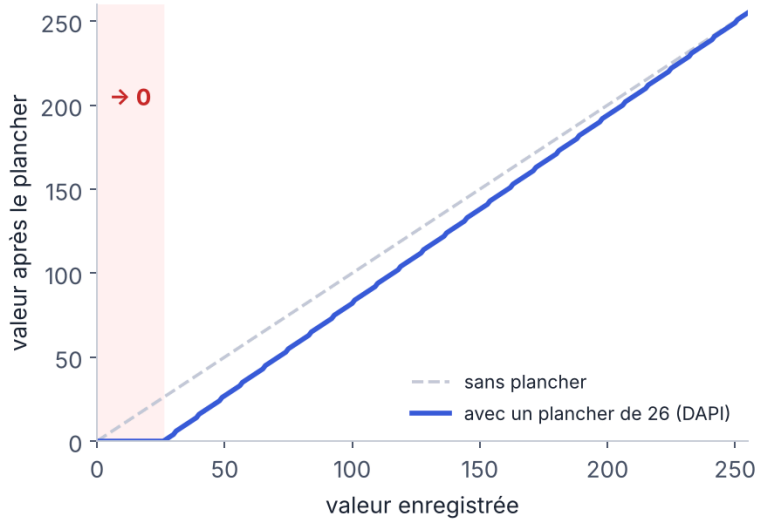


▲ À gauche, la fenêtre min 20 / max 220 ; à droite, l’effet du gamma 0,5 / 1 / 2.

- **Min / Max** : tout ce qui est sous *min* devient noir, au-dessus de *max* pleine couleur.
- **Poignée du milieu** : le niveau affiché à 50 % ; elle règle le gamma (gamma = ln 0,5 ÷ ln *m*).
- **Auto** coupe les 0,5 % les plus sombres et les plus clairs ; **Doux**, **Contraste** et **Réinit.** sont des préréglages.
- **Opacité** : 70 %, 42 % ou 100 % ; **Exposition** : de 0,2× à 5× pour tous les canaux ; **Solo** isole un canal.

- **Instantané** : en mode *Fluorescence (type Imaris)*, changer min, max, gamma, opacité ou couleur, allumer ou éteindre un canal, et les solos, répondent sans recalculer l'image.
- **Flou gaussien** : lissage plan par plan (σ en voxels), disponible aux qualités qui tiennent en une seule texture.
- Un **filtre de simulation des daltonismes** vérifie qu'une figure reste lisible.

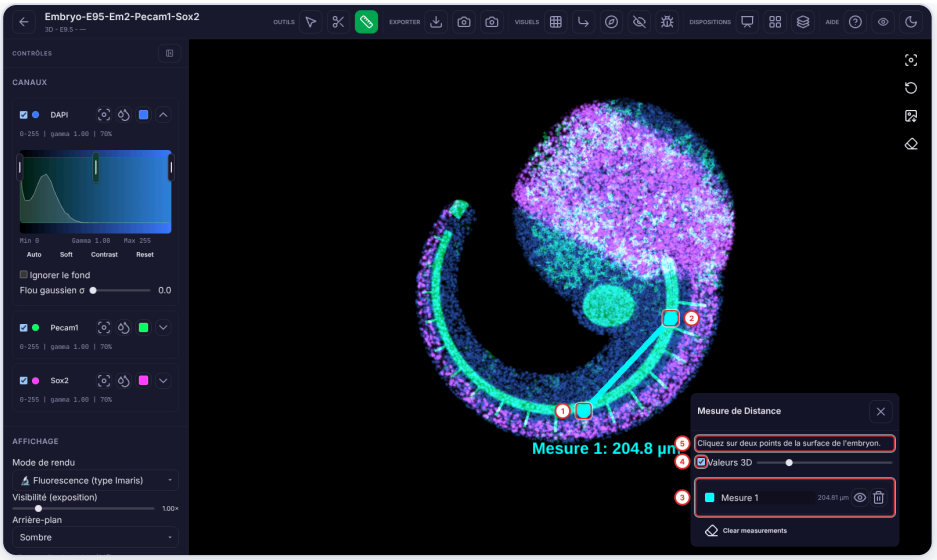
> ⚠ **ATTENTION**
>
> **Un réglage caché.** Avant vos réglages, le viewer met aussi à 0 les valeurs les plus faibles de chaque canal (un « plancher » entre 6 et 48, estimé sur l'histogramme), puis étire le reste. C'est pourquoi le bruit résiduel disparaît à l'écran. → ch. 11



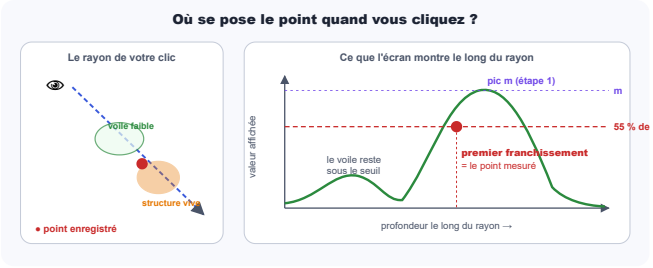
▲ L'effet du plancher d'affichage : estimé à 26 pour le canal DAPI du jeu de démonstration.

11. Mesurer et explorer l'intérieur

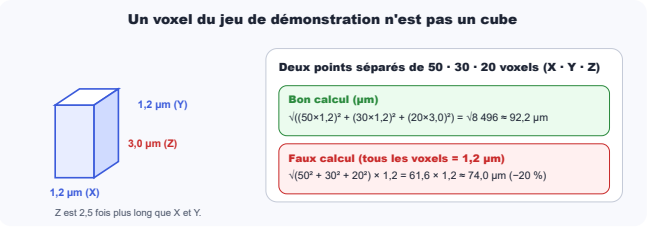
> ⏱ **EN 30 SECONDES**
>
> - La **mesure** se fait en **µm**, avec la vraie taille des voxels, sur la première structure visible sous le clic, **dans l'image affichée**. Sa précision est celle du voxel du niveau affiché.
> - **Coupe oblique** et **navigateur Z-stack** montrent l'intérieur, avec une barre d'échelle exacte.
> - La barre d'échelle de la **vue 3D** n'est exacte qu'à la profondeur du centre de l'échantillon.



▲ Deux points et la distance mesurée : 204,8 µm (jeu de démonstration).



▲ Le point mesuré est la première structure qui atteint 55 % du maximum affiché le long du rayon. Un clic dans le vide est refusé.



▲ On ne compte pas les voxels : en Z, un voxel est ici 2,5 fois plus long.

🔢 **MICRO-EXEMPLE**

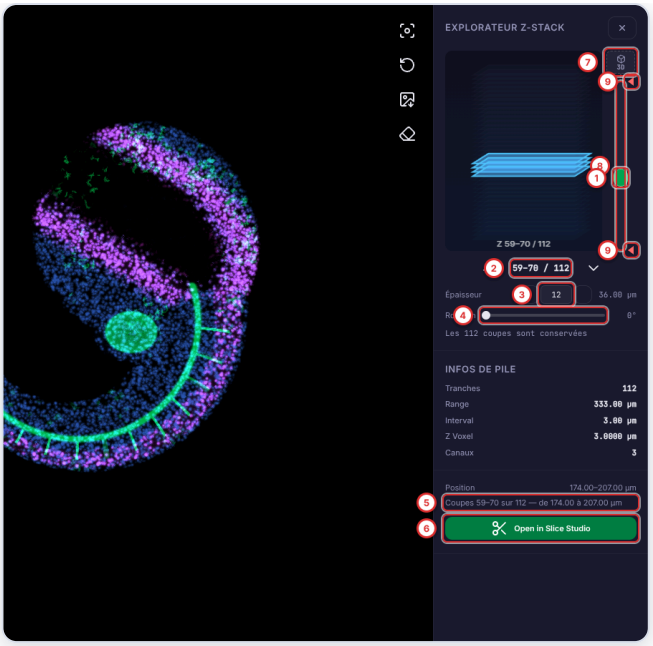
Deux points séparés de 10 voxels en X et 4 voxels en Z, avec des voxels de 1,2 × 1,2 × 3 µm : √((10 × 1,2)² + (4 × 3)²) = √(144 + 144) = **17 µm** — et non √(10² + 4²) ≈ 10,8 « voxels ».

⚠️ **ATTENTION**

Les mesures ne sont pas sauvegardées sur le serveur : fermer l'onglet les efface. Gardez-les en copiant l'adresse de la page (elles voyagent dans le lien) ou en téléchargeant le **CSV des mesures** dans le Centre de téléchargement.

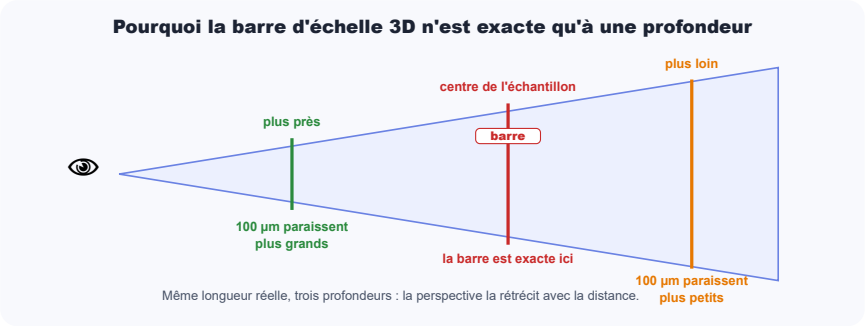


▲ Coupe XY au milieu du volume, avec sa barre d’échelle.



▲ Navigateur Z-stack en mode coupe : 12 coupes (36 µm).

- **Coupe oblique** : un plan dans n’importe quelle direction, affiché en grand avec une barre d’échelle exacte.
- **Navigateur Z-stack** : il couche la pile à plat (face du dessus vers vous), une barre d’épaisseur réglable, des poignées pour rogner le haut et le bas, une rotation de 0 à 360°. Il est exclusif avec la coupe oblique.
- **Axes d’orientation** : la boussole rouge / vert / bleu (R1/R2, G1/G2, B1/B2) définie par l’administrateur. Les axes et la boussole démarrent au **centre de gravité** du spécimen ; un double-clic sur la sphère centrale des axes les recentre.

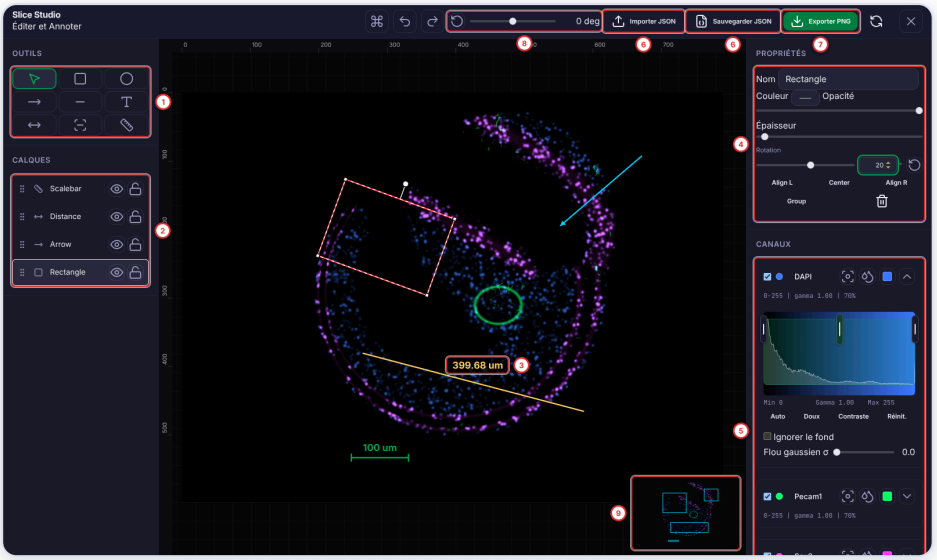


▲ Même longueur, trois profondeurs : la barre de la vue 3D vaut pour le plan du centre de l’échantillon.

12. Faire des figures et exporter

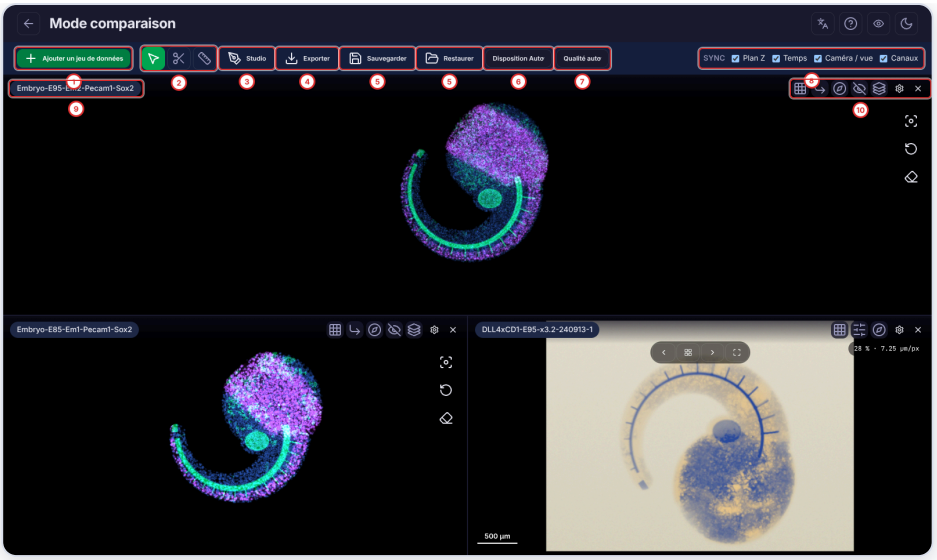
⏱ **EN 30 SECONDES**

- Le **Studio** compose une figure annotée à **pleine résolution** (flèches, textes, distances, barre d’échelle).
- La vue 3D s’exporte **à n’importe quelle taille**, avec fond transparent si besoin.
- **Comparer** met jusqu’à 4 jeux côte à côte, synchronisés.



▲ Le Studio : rectangle tourné, flèche, distance, barre d’échelle et texte, sur une coupe du jeu de démonstration.

- Ouvrir une coupe dans le Studio la recharge **au niveau natif** (plans XY ou plages d’octets), avec une progression.
- Les **couleurs des canaux** se règlent dans le Studio sans toucher aux annotations.
- Export **PNG** (la figure) ou **JSON** (pour la reprendre plus tard) ; au-delà de 5 Mo, l’export est refusé avec un message.
- **Sur un jeu de format 6, les exports sont sans perte, à la résolution affichée, sans question** : la vue 3D, la figure du Studio et *Décomposer par canal*.
- **Sur un jeu de format 5**, l’export demande encore : la vue 3D (**Qualité** : **Actuelle** ou **Sans perte**), le Studio (**Native sans perte** ou **Cette image**). Vos annotations gardent leur place.
- **Décomposer par canal** : un **cadre rouge** sur la vue 3D montre ce que capture l’image principale, dans les dispositions qui en ont une.



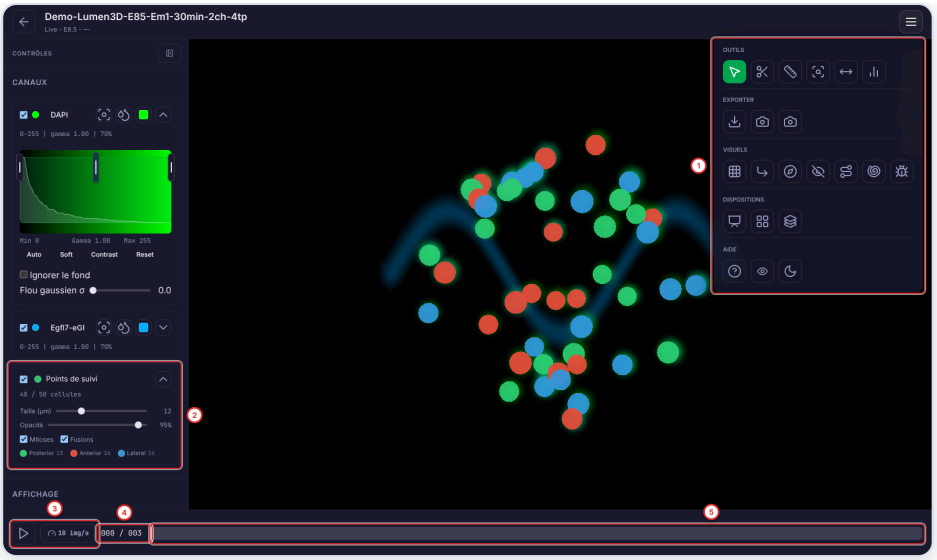
▲ Comparer : deux volumes et une photographie, outils communs et synchronisation (caméra, temps, coupes, canaux).

POUR…	UTILISEZ
une image de la vue 3D, grande et nette	**Exporter la vue** (rendu par tuiles de 2 048 px, fond transparent possible)
une capture rapide	**Capture d’écran**
les fichiers d’origine (`.ims` , TIFF, MIP)	**Centre de téléchargement** (si le dossier `download/` existe)
comparer des embryons ou des canaux	**Comparer** et **Décomposer par canal**
partager exactement ce que vous voyez	l’**adresse de la page** ou l’espace de travail enregistré dans Comparer

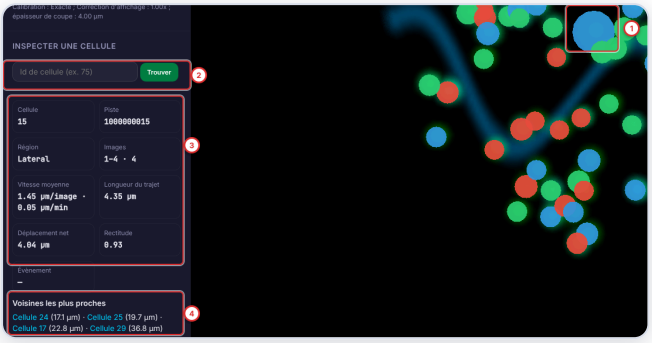
13. Timelapses et suivi cellulaire

⏱ **EN 30 SECONDES**

- La **frise** lit la série de 0,5 à 20 images par seconde ; une barre montre les instants déjà chargés.
- Le **suivi cellulaire** se superpose au volume : cellules, trajectoires, surface de l’embryon.
- Cinq outils d’analyse : trajectoires, surface, inspecteur de cellule, graphiques, distance entre cellules.



▲ Un timelapse suivi (jeu de démonstration, 4 instants) : la couche de suivi et la frise temporelle.



▲ L’inspecteur : métriques, lignée, voisins d’une cellule sélectionnée.



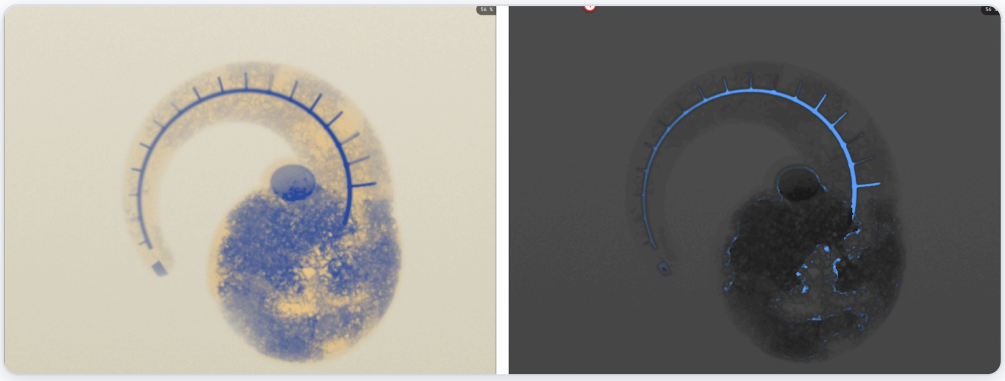
▲ La distance entre deux cellules, suivie dans le temps.

- La lecture **attend** les images qui ne sont pas encore arrivées : elle ne saute jamais un instant en silence.
- Les positions affichées sont les positions **stabilisées** (§ 7) ; les mitoses et fusions sont marquées.
- Les graphiques (population, vitesse, voisins, mitoses) et les tableaux s’exportent. → ch. 12 et 13

14. Les photographies 2D

⏱ EN 30 SECONDES

- Une photographie calibrée par dataset : zoom, barre d’échelle, mesures en µm ou mm.
- **Isoler le marquage** fait ressortir la coloration X-gal (bleue) sur le tissu (jaune).
- Une planche-contact parcourt toutes les photographies de la collection.



▲ Avant / après « Isoler le marquage » : le bleu ressort, le reste passe en gris (photographie de démonstration).

« Isoler le marquage » : trois étapes pour chaque pixel

1 · Où est le tissu jaune ?

« jaunisme » d'un pixel =
(rouge + vert) / 2 − bleu

On le moyenne sur un carré d'environ 40 px de côté (image réduite 4 fois, flou de rayon 5).

fond mat : 2 à 6
tronc coloré large : 6 à 9

→

2 · Le pixel est-il bleu ?

rapport bleu / (rouge + 1)

Tissu non coloré : ≈ 0,6
Cœur très coloré : 2 à 4

force v : 0 si rapport ≤ 1,0
1 si rapport ≥ 1,8, linéaire entre

et seulement si le « jaunisme » local dépasse 5 (sinon v = 0)

→

3 · On repeint

gris = luminosité × 0,35 × (1 − v)

rouge = gris + 90 × v
vert = gris + 160 × v
bleu = gris + 255 × v

v = 0 : gris sombre
v = 1 : bleu vif (90, 160, 255)

Micro-exemple

Pixel (R, V, B) = (60, 90, 170) : 170 / 61 = 2,8 → v = 1, dans un tissu à « jaunisme » 8 : il devient bleu vif.
Pixel (200, 190, 120) : 120 / 201 = 0,6 → v = 0 : il devient gris sombre (0,35 × sa luminosité).

▲ Le calcul, pixel par pixel : rapport bleu / rouge, limité au voisinage du tissu jaune pour ignorer les poussières du fond.

ℹ️ À SAVOIR

La photographie est stockée en **WebP avec perte (qualité 90)** par défaut ; l’option `--lossless` garde chaque pixel exact, utile si l’on veut quantifier le rapport bleu / rouge. La calibration vient des métadonnées du TIFF (ImageJ / Leica) ; sans elle, pas de barre d’échelle. → ch. 13

15. Interpréter correctement ce que l’on voit

📌 **À RETENIR**

- **Les niveaux de gris sont relatifs.** Chaque canal de chaque dataset a sa propre fenêtre (plancher → plafond) : on ne compare pas une intensité entre deux canaux ni entre deux datasets. Dans un timelapse, la fenêtre est **commune à tous les instants** : une baisse d’intensité reste visible, comme dans la réalité.
- **Dans le signal, rien n’est modifié** ; **hors du signal**, le bruit est lissé (médiane 3 × 3 × 3).
- **Un objet plus fin qu’environ 3 voxels** dans une direction est effacé par le nettoyage, même brillant (§ 4.4).
- **8 bits** suffisent pour la morphologie ; pour **quantifier**, repartez du `.ims` d’origine (dans `download/` si l’option a été activée).
- **Les briques non stockées** sont des zones réellement à 0 après nettoyage, pas des données perdues.
- **Le plancher d’affichage** (6 à 48) masque les valeurs les plus faibles à l’écran : baissez *min* ne les fera pas revenir.
- **Au plus 4 canaux** sont affichés en même temps.
- **Une mesure porte sur ce que vous voyez** : la copie Q30 (format 6) et les colonnes avec perte (format 5) changent les valeurs que vous mesurez ; seuls les **exports** lisent toujours les données sans perte, au niveau affiché.
- **Sans calibration** dans le fichier, aucune mesure en µm n’est proposée.
- **La vue 3D est en perspective** : mesurez avec l’outil de mesure ou sur une coupe, pas avec la barre d’échelle 3D.

15.1 Ce que la plateforme ne fait pas

NON DISPONIBLE	POURQUOI / QUE FAIRE
Quantification d’intensité absolue	données en 8 bits fenêtrées : utiliser le `.ims` d’origine
Segmentation, comptage automatique	faire l’analyse dans Imaris ou Fiji
Déconvolution	à faire avant l’export Imaris
Compensation du photoblanchiment	mesurée par le pipeline, mais pas encore appliquée par le viewer
Modifier les voxels	tout est en lecture seule ; seul l’affichage se règle

→ ch. 20 (annexe « Ce que Lumen3D ne fait pas »)

16. Que se passe-t-il si… ?

SITUATION	CE QUE FAIT LA PLATEFORME	CE QUE VOUS POUVEZ FAIRE
L’image est floue au début	le niveau grossier arrive d’abord, le détail suit	attendre la fin de la progression
Une résolution affiche un autre niveau	mémoire graphique insuffisante : niveau plus grossier + message	choisir une résolution plus petite, fermer d’autres onglets 3D
La page dit « Ce navigateur a refusé à la page un contexte 3D (WebGL) »	le navigateur bloque WebGL après un plantage du pilote graphique	fermer complètement le navigateur (sur Android : forcer l’arrêt de Chrome), puis rouvrir la page
La carte graphique « plante »	le viewer recharge la vue à mémoire réduite (3 fois en 2 minutes au plus)	baisser la qualité, recharger la page
Une brique est corrompue	elle est affichée vide et comptée, jamais montrée fausse	prévenir l’administrateur
Le transfert d’import s’arrête	il reprend dès que le réseau revient	reglisser le même dossier si besoin
Le dataset n’est pas dans l’explorateur	il est masqué, ou encore en import	demander à l’administrateur de le publier / l’afficher
Un clic de mesure est refusé	rien de visible sous le curseur	cliquer sur une structure visible
Le flou gaussien est grisé	la qualité choisie est trop grande pour une seule texture	choisir une qualité plus basse

→ ch. 19 (plus de 60 situations détaillées)

17. Les mots à connaître

Voxel

> Le « pixel » d’un volume : un petit pavé, ici 1,2 × 1,2 × 3,0 µm. → ch. 4

Canal

> L’image d’un fluorophore (DAPI, Pecam1…). → ch. 4

Plancher / plafond

> Les deux valeurs qui définissent la conversion en 8 bits : sous le plancher → 0, au-dessus du plafond → 255. → ch. 5

Masque

> Les voxels reconnus comme signal ; seuls ceux hors du masque sont lissés. → ch. 5

Pyramide (niveaux)

> Les versions de plus en plus petites du volume ; le niveau 0 est la pleine résolution. → ch. 6

Brique

> Un cube de 64^3 voxels, l'unité de découpage et de téléchargement. → ch. 7

WebP sans perte

> Le format d'image qui stocke les briques en gardant chaque valeur exacte. → ch. 6

Q30

> Le WebP à qualité 30 : la copie que l'écran affiche au format 6. Erreur moyenne mesurée : 1,5 niveau de gris sur 255. → ch. 6 et 9

MIP

> Projection d'intensité maximale : on garde la valeur la plus forte le long d'un axe. → ch. 7 et 9

Lancer de rayons

> La méthode de dessin de la vue 3D, exécutée par la carte graphique. Pour chaque pixel de l'écran, un rayon traverse le volume par pas réguliers (environ un échantillon par voxel) ; chaque échantillon est lu dans la texture 3D et cumulé avec les précédents, d'avant en arrière, jusqu'à la sortie du volume ou, dans le mode Fluorescence par défaut, jusqu'à une opacité presque totale. → ch. 9

Gamma

> L'exposant qui éclaircit (< 1) ou assombrit (> 1) les tons moyens. → ch. 11

Histogramme

> Le graphique qui compte combien de voxels ont chaque valeur. → ch. 11

Stabilisation

> Le déplacement rigide qui compense la dérive de l'embryon dans un timelapse. → ch. 8 et 13

Studio

> L'atelier de figures annotées à pleine résolution. → ch. 12 et 13

Format de données

> Le numéro de version de l'organisation des fichiers d'un dataset (1 à 6). → ch. 17

Matrice (de qualité)

> En format 5, le tableau « résolution × compression » du panneau de qualité : chaque case est une façon d'afficher le volume. → ch. 10

📖 **POUR ALLER PLUS LOIN**

Documentation complète (« Lumen3D, de A à Z ») : 21 chapitres, plus de 350 pages, chaque étape illustrée. **Guide de l'administrateur** : tous les écrans du panneau, pas à pas. Les deux sont publiés dans l'onglet *Documentation* du panneau d'administration.